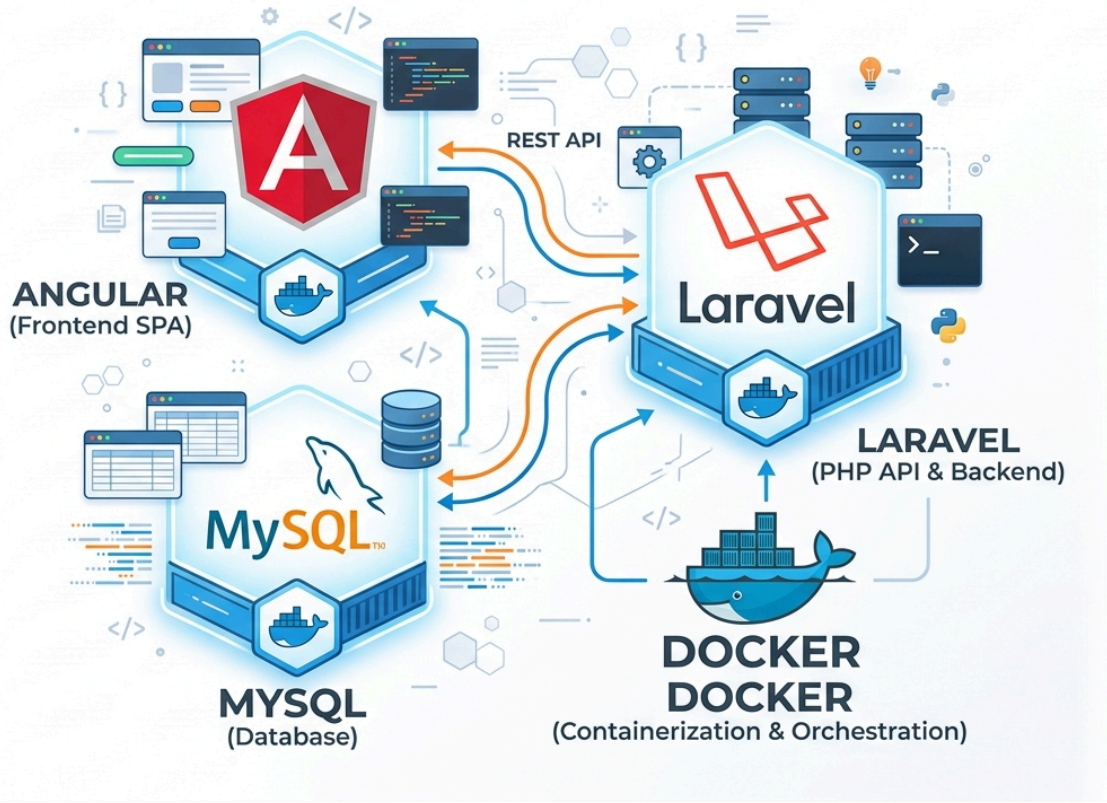
DOCUMENTACIÓ TÈCNICA DESPLEGAMENT D'APLICACIONS WEB



Alumnes: Albert Prats, Oriol Valentines i Marc Valentines
Curs: 2n Desenvolupament d'aplicacions Web
Data: Divendres, 1 de maig de 2026

índex:

1 Descripció del projecte....4
2 Arquitectura de l'aplicació.... 5
2.1 Visió General:....5
2.1.1 Frontend:....5
2.1.2 Backend:.... 6
2.1.3 Base de Dades:....6
2.2 Arquitectura Xuxemons Visual:.... 7
3 Contenidors Docker:....7
3.1 Arquitectura Docker....9
4 Estructura de les Carpetes del Projecte:.... 9
4.1 Capa de Presentació (Frontend)....9
4.2 Capa de Lògica de Negoci i API (Backend).... 10
4.3 Capa de Persistència de Dades (Base de dades).... 10
4.4 Diagrama Capes del Projecte....11
4.5 Diagrama d'Arquitectura per Capes.... 11
5 Tecnologies Utilitzades:.... 12
5.1 Infraestructura.... 13
5.1.1 Justificació:....13
5.2 Capa de Base de Dades (Persistència).... 13
5.2.1 Justificació:....13
5.3 Capa de Backend (Lògica de Negoci i API REST).... 13
5.3.1 Justificació:....14
5.4 Eloquent ORM i Sistema de Migracions....14
5.4.1 Justificació:....14
5.5 Autenticació i Seguretat: JWT Auth (tymon/jwt-auth)....14
5.5.1 Justificació:....14
5.6 Temps real: Pusher PHP Server (pusher/pusher-php-server)....15
5.6.1 Justificació:....15
5.7 Programació reactiva: RxJS....15
5.7.1 Justificació:....15
5.8 Auth0 Angular JWT (@auth0/angular-jwt).... 16
5.8.1 Justificació:....16
6 Requisits desplegament:....16
6.1 Requisits de Programari (Software).... 16
6.2 Requisits de Xarxa (Ports i Connectivitat)....16
6.3 Configuració i Variables d'Entorn.... 17
7 Procés d'instal·lació:.... 17
7.1 Pas 1: Preparació del servidor.... 18
7.2 Pas 2: Obtenir el codi font....18
7.3 Pas 3: Configuració de l'entorn.... 18
7.4 Pas 4: Preparació de volums i permisos....19
7.5 Pas 5: Construcció de les imatges....19
8 Posada en marxa:.... 20

9 Variables d’entorn utilitzades:..... 20
9.1 Configuració General..... 21
9.2 Base de Dades (MySQL)..... 21
9.3 Websocket (Pusher)..... 22
9.4 Autenticació (JWT)..... 23

1 Descripció del projecte

Aquest projecte, anomenat Xuxemons, consisteix en el desenvolupament d'una aplicació web pensada per oferir una experiència interactiva, divertida i social als usuaris. La idea principal és crear un univers digital on cada jugador pugui col·leccionar unes criatures anomenades Xuxemons, cuidar-les, fer-les créixer i interactuar amb altres jugadors.

A través de la plataforma, els usuaris no només gestionen els seus propis recursos, com ara xuxes (dolços) o inventari, sinó que també construeixen una col·lecció personal i prenen decisions que afecten l'evolució dels seus Xuxemons. A més, el joc incorpora mecàniques dinàmiques com malalties, vacunes i recompenses diàries, que aporten un toc d'estratègia i fan que l'experiència sigui més rica i variada.

El projecte també té una vessant social important: els jugadors poden afegir amics, intercanviar elements, xatejar en temps real i fins i tot competir en batalles, creant així una comunitat activa dins de l'aplicació.

Des del punt de vista tècnic, es tracta d'una aplicació moderna basada en una arquitectura client-servidor, amb un frontend dinàmic desenvolupat amb Angular i un backend robust amb Laravel, tot desplegat mitjançant Docker. També s'hi apliquen bones pràctiques de seguretat, com l'autenticació amb tokens JWT, i es tenen en compte aspectes com l'accessibilitat i el SEO.

En definitiva, Xuxemons no és només una aplicació web, sinó una combinació d'aprenentatge tècnic i creativitat, on es posen en pràctica coneixements de desenvolupament, disseny i treball en equip per construir un producte complet i funcional.

2 Arquitectura de l’aplicació

2.1 Visió General:

Aquesta és una aplicació full-stack, dissenyada amb tres capes, que són el frontend, el backend i MySQL. Alhora, s’utilitza Docker per al desplegament de l’aplicació en qüestió; aquest permet un desplegament ràpid mitjançant Docker Compose.

Frontend (capa de presentació)**: desenvolupat en Angular 18.**
Backend (capa de lògica i API)**: desenvolupat en Laravel 12.**
Base de dades (capa de dades)**: implementada amb MySQL.**

En aquesta aplicació s’utilitza una arquitectura basada en WebSockets, a través de Pusher, per a la comunicació bidireccional, la qual s’empra en el xat o en la lluita de Xuxemons (sistema de combat en línia).

2.1.1 Frontend:

El frontend proporciona una interfície d’usuari adaptativa i reactiva amb la qual els jugadors poden interactuar per gestionar els seus Xuxemons, consultar la Xuxedex i participar en batalles…

Tecnologia principal: Angular 18 (TypeScript, HTML i SCSS/CSS).
Gestió de l’estat: RxJS per gestionar els fluxos de dades asíncrons i modelar la reactivitat dins dels components.
Autenticació i seguretat: *JSON Web Tokens (JWT)* integrats amb @auth0/angular-jwt. El sistema emmagatzema un token al *localStorage* per mantenir la sessió del jugador de manera segura durant hores.
Comunicació en temps real: interfície integrada amb *pusher-js* i *laravel-echo* per escoltar esdeveniments del backend a l’instant. Això és indispensable per notificar als clients els canvis d’estat asíncrons, com ara “jugador llest” o “missatge enviat” a l’arena de combat.
Infraestructura de xarxa local: contenidor frontend-angular, publicat al port local 4200.

2.1.2 Backend:

El backend actua com el nucli central ("cervell") del joc. En lloc de generar vistes web (HTML/Blade), proporciona una API REST aïllada (headless) perquè la consumeixi el frontend, garantint una alta separació de responsabilitats.

Tecnologia principal: framework Laravel 12 (PHP 8.2+).
Patró arquitectònic: Model-Vista-Controlador (MVC), enfocat a controladors d'API que retornen les peticions de manera unificada en format JSON.
Capa ORM: ús de l'Eloquent ORM de Laravel per interactuar de manera fluida amb la capa de persistència de dades mitjançant models i implementar el control de canvis d'esquema a través de fitxers de migració (migrations) i seeders.
Autenticació i autorització: *tymon/jwt-auth* a nivell d'API i middleware que protegeixen els endpoints sensibles i d'administració (admin).
Emissió d'esdeveniments: paquet *pusher-php-server*, encarregat d'emetre (broadcasting) esdeveniments cap als comptes de Pusher del frontend per mantenir l'estat de dos oponents sempre sincronitzat en el context multijugador.
Infraestructura de xarxa local: contenidor backend-laravel, publicat al port local *8000*.

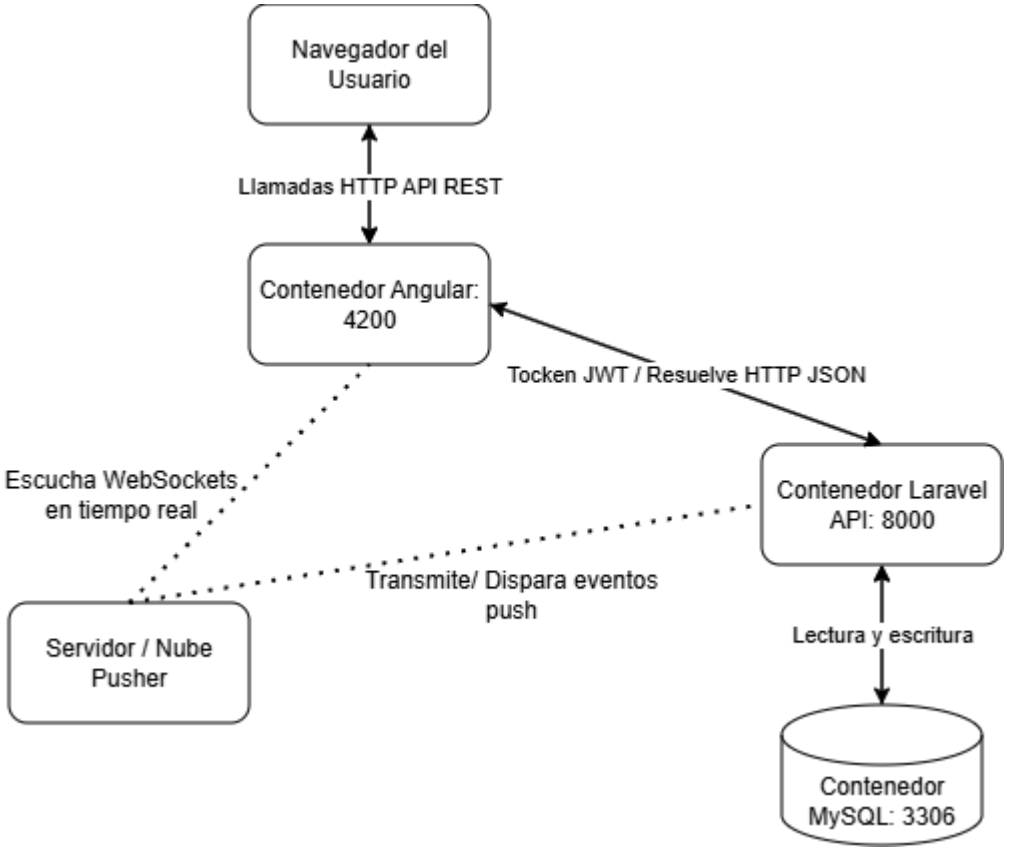
2.1.3 Base de Dades:

L'emmagatzematge de la informació gestiona la persistència i la relació estricta de l'entorn relacional, permetent una alta integritat i rendiment.

Tecnologia principal: MySQL 8+.
Estructura relacional: taules connectades mitjançant claus foranes, que allotgen la matriu d'usuaris, l'inventari en propietat, el catàleg de la Xuxedex, els registres d'infeccions o curacions i l'historial o l'estat actiu de les batalles.
Persistència (Docker Volumes): les dades físiques no s'emmagatzemen en el contenidor efímer, sinó en el volum extern (db_volume), garantint que tota la informació (com els Xuxemons capturats i el progrés de la partida) es mantingui disponible encara que els contenidors es recreïn o es reiniciïn.

Infraestructura de xarxa local: contenidor db-mysql, que utilitza el port estàndard *3306*.

2.2 Arquitectura Xuxemons Visual:



3 Contenidors Docker:

Aquest és el diagrama d'arquitectura; està basat exactament en el fitxer *docker-compose.yml*. Mostra els contenidors que estan configurats, la xarxa que comparteixen, els seus volums i les dependències.

fullstack-network (xarxa compartida): Tots els serveis estan connectats a aquesta xarxa en mode *bridge*, cosa que significa que poden comunicar-se internament resolent el nom del servei (exemple: *db* o *backend*).

frontend-angular (servei: frontend):

- **Tecnologia:** Angular.
- **Port:** Exposa el *4200* a la màquina local.
- **Dependències:** Configurat amb *depends_on: backend*, cosa que significa que Docker Compose espera que el backend arrenqui abans d'iniciar aquest.
- **Volums:** Sincronitza la carpeta *./frontend* amb */app* dins del contenidor.

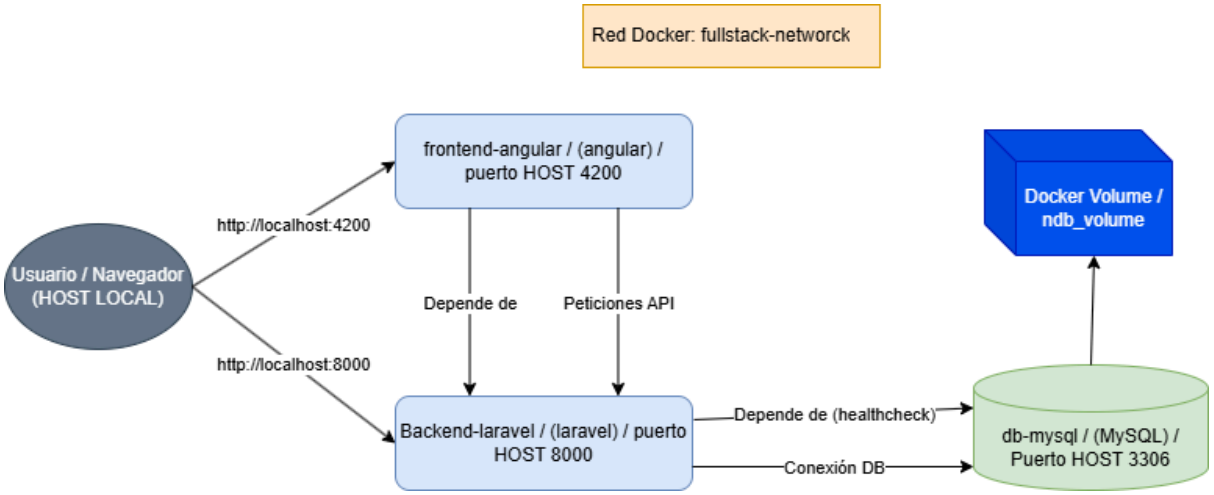
backend-laravel (servei: backend):

- **Tecnologia:** Laravel (PHP).
- **Port:** Exposa el *8000* a la màquina local.
- **Dependències:** Configurat amb *depends_on: db* (*service_healthy*). El backend no arrenca fins que el contenidor de MySQL supera correctament la prova de «ping».
- **Volums:** Sincronitza *./backend* amb */app*.
- **Connexió DB:** Utilitza l'host db al port *3306* amb la base de dades xuxemons_db i l'usuari *xuxemons_admin*.

db-mysql (servei: db):

- **Tecnologia:** MySQL.
- **Port:** Exposa el *3306* a la teva màquina local.
- **Healthcheck:** Fa un *mysqladmin ping* cada 5 segons per informar la resta de serveis de quan està llest.
- **Volums:** Utilitza un volum de Docker gestionat anomenat *db_volume* per assegurar que les dades no s'esborrin quan el contenidor s'aturi o es reinici.

3.1 Arquitectura Docker



4 Estructura de les Carpetes del Projecte:

4.1 Capa de Presentació (Frontend)

Tecnologia: Angular (TypeScript, HTML, CSS)
Directori principal: /frontend

Aquesta és la interfície d'usuari amb la qual interactuen les persones. S'encarrega exclusivament de la lògica de presentació, la gestió de l'estat visual (com els inicis de sessió, les pàgines i l'inventari) i l'encaminament del costat del client.

src/app/: Aquí resideixen els components d'Angular (com pagina-principal), serveis, interfícies i utilitats.

Comunicació: No accedeix a la base de dades de manera directa. Envia peticions HTTP (generalment en format JSON) cap a la capa de Backend per consultar o modificar dades.

Docker: S'executa en el contenidor frontend-angular (exposat al port 4200) i utilitza un volum per sincronitzar en temps real el codi font (*/frontend:/app*), cosa que permet un desenvolupament fluid.

4.2 Capa de Lògica de Negoci i API (Backend)

Tecnologia: Laravel (PHP)
Directori principal: /backend

Aquesta capa actua com el “cervell” centralitzat del projecte. Proporciona els endpoints d’una API RESTful que el Frontend consumeix.

Responsabilitats: Validar les dades d’entrada, aplicar les regles de negoci (per exemple, validació d’inventari, apilament d’objectes, creació d’usuaris), gestionar l’autenticació i processar les peticions.

Estructura interna de Laravel:

- *app/:* Conté els controladors i models.
- *routes/:* Defineix les rutes de l’API (generalment a api.php).
- *database/:* Gestiona les migracions i els seeders per estructurar i inicialitzar la base de dades de manera automatitzada.

Comunicació: Rep peticions des del Frontend, interactua amb la base de dades mitjançant l’ORM (Eloquent), formata la resposta (JSON) i la retorna al client.

Docker: S’executa al contenidor backend-laravel (port 8000). Igual que el Frontend, utilitza volums per al codi en desenvolupament.

4.3 Capa de Persistència de Dades (Base de dades)

Tecnologia: MySQL
Directori principal: /db (conté principalment la configuració de Docker)

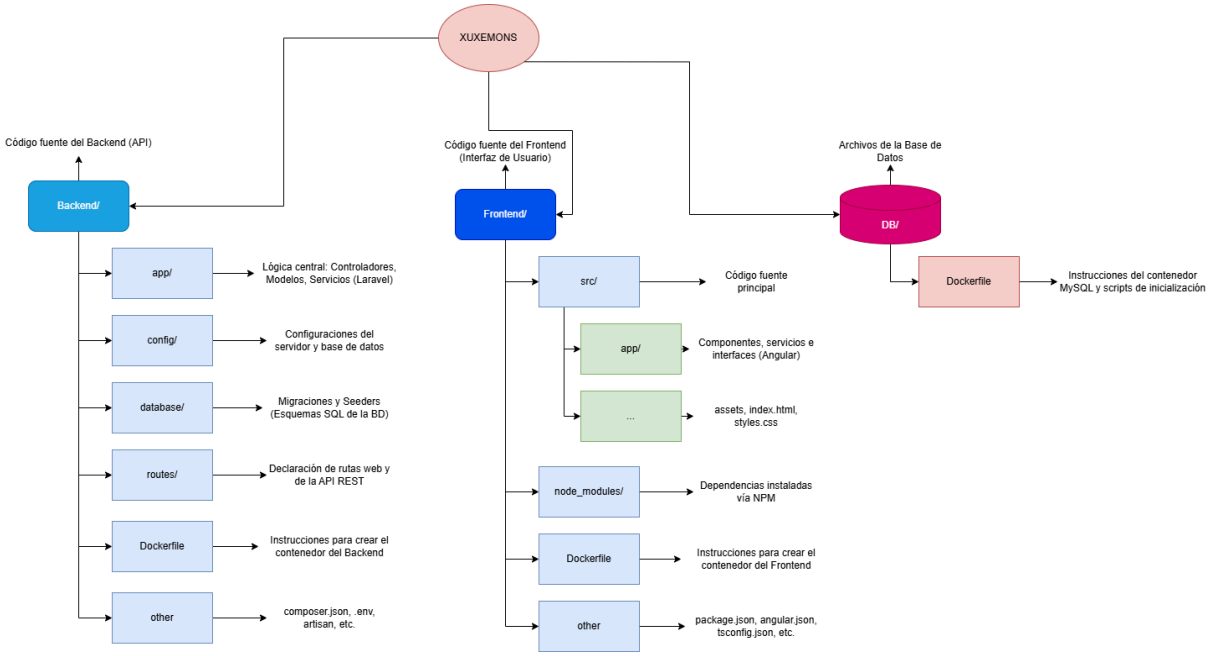
És el repositori central on s’emmagatzema de manera permanent tota la informació de l’aplicació (usuaris, inventari, progrés, configuracions, etc.).

Responsabilitat: Garantir la integritat, seguretat i disponibilitat de les dades.

Docker: S'executa al contenidor db-mysql (port 3306).

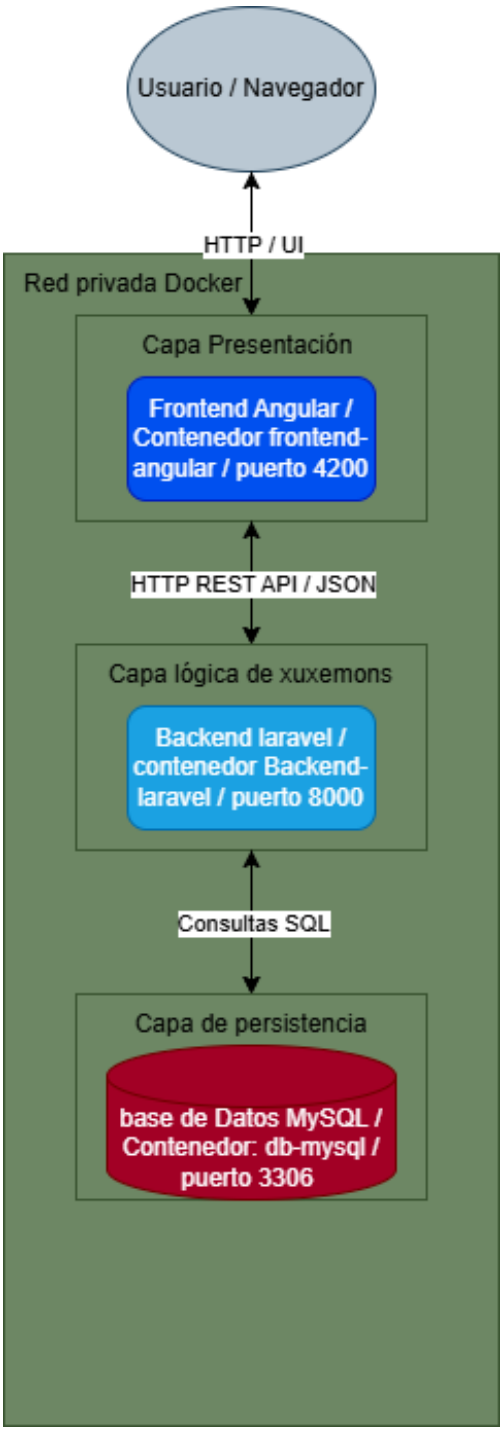
Volum segur: Utilitza un volum de Docker (db_volume:/var/lib/mysql), assegurant que, encara que el contenidor s'aturi o es reiniciï, les dades de la base de dades no es perden.

4.4 Diagrama Capes del Projecte



4.5 Diagrama d'Arquitectura per Capes

A continuació, es mostra un diagrama que il·lustra com es comuniquen les diferents parts del sistema dins de la xarxa definida per Docker (fullstack-network):



5 Tecnologies Utilitzades:

A continuació, es detalla de manera exhaustiva el stack tecnològic implementat en el projecte Xuxemons.

5.1 Infraestructura

Docker & Docker Compose
Ús: Contenidorització i orquestració de la base de dades, el backend i el frontend.

5.1.1 Justificació:

Estandardització: En encapsular l'aplicació en contenidors, s'elimina el problema de "al meu ordinador funciona". Les versions de PHP, Node i MySQL són idèntiques en tots els entorns.
Xarxes privades: Docker crea la xarxa *fullstack-network*, aïllant la comunicació entre el backend i la base de dades perquè no sigui accessible directament des de l'exterior, cosa que millora la seguretat.

5.2 Capa de Base de Dades (Persistència)

MySQL

Ús: Sistema de gestió de bases de dades relacional (RDBMS).

5.2.1 Justificació:

- El model del joc requereix relacions fortes i estructurades (per exemple: un usuari té molts Xuxemons, un usuari té un inventari). MySQL garanteix aquestes restriccions d'integritat referencial.

- Alta compatibilitat i integració nativa amb el motor de bases de dades de Laravel

5.3 Capa de Backend (Lògica de Negoci i API REST)

PHP 8.2 i Laravel 12

Ús: Llenguatge principal de servidor i framework web.

5.3.1 Justificació:

PHP 8.2 ofereix un excel·lent rendiment i tipatge fort. Laravel 12 és un dels frameworks més avançats de PHP, proporcionant una arquitectura neta (MVC), seguretat robusta (prevenció d'injeccions SQL i atacs CSRF) i un ecosistema ampli per crear APIs escalables de manera ràpida.

5.4 Eloquent ORM i Sistema de Migracions

Ús: Mapeig objecte-relacional (ORM) i control de versions de la base de dades.

5.4.1 Justificació:

Eloquent: Permet interactuar amb la base de dades MySQL utilitzant objectes de PHP en lloc d'escriure consultes SQL directes. Això fa que el codi sigui molt més llegible i menys vulnerable a injeccions.

Migracions / Seeders: Actuen com un control de versions de l'estructura de la base de dades, permetent construir i omplir la base de dades (amb usuaris inicials, xuxemons per defecte i objectes) automàticament en el desplegament.

5.5 Autenticació i Seguretat: JWT Auth (tymon/jwt-auth)

Ús: Gestió de sessions mitjançant JSON Web Tokens (JWT).

5.5.1 Justificació:

Les APIs REST han de ser stateless (sense estat). En lloc d'emmagatzemar una sessió al servidor, el backend signa un token que s'entrega al client (Angular) després del login. Aquest token garanteix

que l'usuari és qui diu ser en cada petició, cosa ideal per a arquitectures desacoblades.

5.6 Temps real: Pusher PHP Server (pusher/pusher-php-server)

Ús: Gestió d'esdeveniments mitjançant WebSockets.

5.6.1 Justificació:

Per a un joc és essencial que es produeixin esdeveniments en "temps real" (per exemple: notificacions push, avisos de combat, actualització d'estadístiques en viu). Pusher facilita la infraestructura de WebSockets, permetent que Laravel "emeti" informació al navegador del jugador de manera instantània, hem utilitzat aquest 'WebSocket' sobre altres degut a les limitacions que ens ha provocat el propi Docker, els websockets 'self-hosted' com Laravel Reverb aumenten considerablement el consum de memòria RAM, fent que el projecte tingui un rendiment inferior, a diferència de Pusher, que es una plataforma cloud sense consumir més recursos dels necessàris.

5.7 Programació reactiva: RxJS

Ús: Gestió de fluxos de dades asíncrones.

5.7.1 Justificació:

Angular està construït sobre RxJS. Permet gestionar peticions HTTP, esdeveniments de l'usuari i estats canviants mitjançant Observables. Això és essencial per a un joc on múltiples fluxos de dades (per exemple: càrrega d'un nivell + recepció d'un missatge de xat) poden ocórrer simultàniament.

5.8 Auth0 Angular JWT (@auth0/angular-jwt)

Ús: Interceptor i descodificador de tokens al client.

5.8.1 Justificació:

Facilita la feina al frontend. Permet injectar automàticament el token d'autorització en cada petició HTTP cap a Laravel de manera transparent, i comprovar fàcilment si la sessió del jugador ha expirat.

6 Requisits desplegament:

6.1 Requisits de Programari (Software)

El servidor on s'allotgi l'aplicació ha de tenir instal·lat el següent programari bàsic, ja que l'arquitectura es basa principalment en la tecnologia de contenidors.

Sistema Operatiu: Es recomana una distribució de Linux orientada a servidors (com Ubuntu Server 20.04/22.04 LTS o Debian 11/12). També és compatible amb Windows Server, però Linux és preferible per a entorns Docker.
Gestor de contenidors: Docker Engine (versió 20.10 o superior).
Orquestrador local: Docker Compose (versió V2) per gestionar el desplegament simultani de tots els serveis (Frontend, Backend, Base de dades i Nginx).
Control de versions: Git (necessari si es fa el desplegament clonant el repositori directament al servidor).

6.2 Requisits de Xarxa (Ports i Connectivitat)

El servidor ha de tenir accés a internet per descarregar les imatges de Docker i instal·lar dependències. A més, s'han d'habilitar les següents regles al tallafocs (Firewall) perquè l'aplicació sigui accessible:

Port 80 (TCP): Necessari per al trànsit web HTTP. El servidor Nginx escoltarà per aquest port per servir el Frontend i fer de Reverse Proxy cap al Backend.
Port 443 (TCP): (Recomanat) Necessari per al trànsit web segur HTTPS, si es configuren certificats SSL.
Port 8000 (TCP): Si es desitja fer peticions al Backend de forma directa sense passar per Nginx (només en entorns de desenvolupament o configuracions específiques).
Port 3306 (TCP): Només cal obrir-lo si es requereix accés remot a la base de dades MySQL per tasques d'administració (en producció, es recomana mantenir-lo tancat a l'exterior i accedir-hi només des de dins de la xarxa de Docker).

6.3 Configuració i Variables d'Entorn

Abans de llançar el desplegament, cal disposar dels fitxers de configuració d'entorn. S'ha de crear o configurar un fitxer .env (o diversos fitxers, depenent de l'estructura) a l'arrel del projecte que contingui:

Credencials de la base de dades: Nom de l'usuari, contrasenya, nom de la base de dades (xuxemons_db) i el host (que serà el nom del contenidor de MySQL).
Claus de seguretat: Clau secreta JWT (JWT_SECRET) per a l'autenticació i manteniment de les sessions dels usuaris.
Rutes de les API: URL on el Frontend anirà a buscar el Backend (/api/).
Ports interns: Definició dels ports on escoltarà cada servei de Docker internament.

7 Procés d'instal·lació:

Aquest apartat detalla els passos necessaris per instal·lar i preparar l'entorn de l'aplicació Xuxemons al servidor abans de la seva posada en marxa. Com que el projecte està basat en contenidors Docker, el procés s'ha simplificat per garantir que es pugui replicar fàcilment en diferents entorns.

7.1 Pas 1: Preparació del servidor

Abans de descarregar l'aplicació, assegura't de complir els requisits previs:
Tenir instal·lats Docker i tenir instal lat el composer al sistema.
Disposar de permisos d'administrador (root).
Tenir instal·lat Git per poder clonar el repositori del codi font (opcional: CMD).

7.2 Pas 2: Obtenir el codi font

Per instal·lar l'aplicació, el primer pas és clonar el repositori del projecte al servidor on es farà el desplegament. Accedeix al directori on vulguis guardar el projecte i executa la següent comanda:

- **git clone https://github.com/praatts/xuxemons.git**
- **cd xuxemons**

```
C:\Users\albertprba>git clone https://github.com/praatts/xuxemons
Cloning into 'xuxemons'...
remote: Enumerating objects: 4752, done.
remote: Counting objects: 100% (133/133), done.
remote: Compressing objects: 100% (89/89), done.
remote: Total 4752 (delta 67), reused 84 (delta 44), pack-reused 4619 (from 1)
Receiving objects: 100% (4752/4752), 28.47 MiB | 21.63 MiB/s, done.
Resolving deltas: 100% (3005/3005), done.

C:\Users\albertprba>
```

7.3 Pas 3: Configuració de l'entorn

Un cop clonat el repositori, és essencial configurar les variables d'entorn. Al directori arrel del projecte hi haurà un fitxer de plantilla anomenat .env.example. Cal fer-ne una còpia anomenada .env i omplir els valors segons l'entorn:

- **cp .env.example .env**

7.4 Pas 4: Preparació de volums i permisos

L'aplicació necessitarà guardar dades de forma persistent (especialment la base de dades MySQL). Docker Compose s'encarregarà de crear els volums automàticament. No obstant això, assegura't que l'usuari amb què executes Docker tingui permisos suficients a la carpeta del projecte per evitar problemes d'escriptura quan es generin arxius o logs.

7.5 Pas 5: Construcció de les imatges

Finalment, abans de posar en marxa l'aplicació, és recomanable compilar o "construir" les imatges de Docker perquè incloguin els últims canvis del codi:

- docker compose up –build -d

```
C:\Users\albertprba\xuxemons>docker compose up --build -d
#1 [internal] load local bake definitions
#1 reading from stdin 1.48kB 0.0s done
#1 DONE 0.0s
```

al finalitzar la comanda, mostrará el estat dels contenidors:

```
#37 [db] resolving provenance for metadata file
#37 DONE 0.1s
[+] up 8/8
 ✔ Image fullstack-xuxemons-frontend            Built                                                    28.0s
 ✔ Image fullstack-xuxemons-db                  Built                                                    28.0s
 ✔ Image fullstack-xuxemons-backend             Built                                                    28.0s
 ✔ Network fullstack-xuxemons_fullstack-network Created                                                   0.1s
 ✔ Volume fullstack-xuxemons_db_volume          Created                                                   0.0s
 ✔ Container db-mysql                           Healthy                                                  36.4s
 ✔ Container backend-laravel                    Started                                                  35.0s
 ✔ Container frontend-angular                   Started                                                  33.6s

C:\Users\albertprba\xuxemons>_
```

Aquest procés llegirà els Dockerfile del Frontend, del Backend Node.js i de Nginx, descarregant les dependències necessàries i preparant l'entorn perquè estigui llest per arrencar.

8 Posada en marxa:

Per posar el projecte en marxa, amb el procés d'instal·lació no es suficient, ja que el frontend arranca correctament, però s'han de seguir una sèrie de passos per fer funcionar el backend.

Primer, haurem de copiar el arxiu .env de exemple per assignar les nostres **variables d'entorn** necessàries per fer funcionar el projecte correctament amb la comanda, per això, copiarem l'arxiu d'exemple proporcionat al directori de **backend** i asignarem el nom **.env**

```
C:\Users\albertprba\xuxemons\backend>copy .env.example .env
        1 archivo(s) copiado(s).

C:\Users\albertprba\xuxemons\backend>
```

Una vegada assignades les variables d'entorn al arxiu .env, haurem d'executar les següents comandes a la terminal del contenedor de backend.

- **php artisan migrate:fresh --seed**
- **php artisan key:generate**
- **php artisan jwt:secret**

Aquestes comandes permeten omplir la base amb usuaris de prova (junt amb l'admin) i crear els xuxemons i objectes que té el joc, a més d'assignar la clau per l'autenticació JWT i generar la clau encriptada de la base de dades.

Una vegada fet tot això el joc ja serà completament funcional.

9 Variables d'entorn utilitzades:

L'aplicació Xuxemons fa ús de variables d'entorn per gestionar la configuració de forma segura. Aquestes variables permeten adaptar el comportament del Frontend, del Backend Node.js i de la Base de dades

sense necessitat de modificar el codi font. Generalment s'emmagatzemen al fitxer *.env* a l'arrel del projecte.

9.1 Configuració General

Nom de variable	Valor actual	Descripció
APP_NAME	laravel	Nom de l'aplicació.
APP_ENV	local	Entorn d'execució actual (desenvolupament local).
APP_KEY	exemple (base64:F7DXCqmE98Xgb/P9e/vWBj4q6gAXWeAetHgQuD+yzNs=)	Clau de xifratge principal de l'aplicació.
APP_DEBUG	true	Habilita el mode de depuració per veure errors complets.
APP_URL	http://localhost	URL principal de l'aplicació.

9.2 Base de Dades (MySQL)

Nom de variable	Valor actual	Descripció
DB_CONNECTION	mysql	Tipus de base de dades utilitzada.
DB_HOST	db	Host de la base de dades (apunta al nom del contenidor de Docker).

DB_PORT	3306	Port pel qual es connecta MySQL.
DB_DATABASE	xuxemons_db	Nom de la base de dades on es guarden els Xuxemons.
DB_USERNAME	xuxemons_admin	Usuari amb accés a la base de dades.
DB_PASSWORD	1234	Contrasenya d'accés a la base de dades.

9.3 Websocket (Pusher)

App keys

If a token is compromised, you can create a new key/secret pair.
You should delete the old token once you've updated your app.

Created less than a minute ago Copy

```
app_id = "2148231"
key = "fbf7582bc2d86b5453d5"
secret = "f863e83feb07b0a27995"
cluster = "eu"
```

Aquestes variables están inhabilitados, són un exemple.+

Nom de variable	Valor actual	Descripció
BROADCAST_CONNECTION	pusher	Servei encarregat de transmetre (broadcast) els esdeveniments.
PUSHER_APP_ID		ID del projecte donat d'alta a Pusher.

PUSHER_APP_KEY		Clau pública per connectar el client a Pusher.
PUSHER_APP_SECRET		Clau secreta (privada) utilitzada pel backend.
PUSHER_APP_CLUSTER	eu	Zona geogràfica (clúster europeu) on s'allotja el servei.
PUSHER_APP_PORT	443	Port de connexió segura.
PUSHER_APP_SCHEME	https	Protocol utilitzat per comunicar-se amb Pusher.

Per obtenir els valors de les variables d'entorn:

- **PUSHER_APP_ID**
- **PUSHER_APP_KEY**
- **PUSHER_APP_SECRET**

Us heu de registrar a https://pusher.com/ i crear un canal, al crear-la obtindreu aquestes variables d'entorn.

9.4 Autenticació (JWT)

Nom de variable	**Valor actual**	**Descripció**
JWT_SECRET	Es genera automàticament al llençar la comanda php artisan jwt:secret	Clau secreta amb què se signen els tokens de sessió dels usuaris.
JWT_ALGO	HS256	Algorisme

		d'encriptació utilitzat per defecte.